

Day 12 — Traceability Check & Self-Review

Traceability audit across all three RTLs, the two feature tables built to close the gap it found, closure metrics, and a rubric self-review.

12.1 Traceability Check — Results

Running through each RTL, per the assignment's instruction:

FIFO — chain unbroken

All 8 features (Day 9) → all have Req-IDs (Day 9) → all have a checker, stimulus, and priority (Day 10) → all have a coverage item (Day 11). Every coverage item traces back to exactly one Req-ID. **No gaps.**

Sequence Detector — gap found

Day 11 built a coverage model (4 cover points: state, transition, **detected**, overlap-gap) directly from the Day 7 corner-case list — but **none of the Day 7 corner cases, and no basic functional behavior, ever got a Req-ID, checker type, stimulus type, or priority assigned.** The coverage model exists in a vacuum, not linked to anything.

Traffic Light — gap found

Day 10 wrote 3 plain-English assertions directly from the design's known behavior — but again, **no Req-IDs, no priorities, and no formal coverage items** were ever built for this design. **ISSUE-D5-02** (the actual reset bug found on Day 5) doesn't have a permanent regression test tracked anywhere in the verification plan — it was fixed and re-simulated once, but nothing *requires* it to be checked again in the future the way **RST_001** requires it for the FIFO.

This gap is logged as **ISSUE-D12-01** in the Issue Log, **and closed below** — not just noted and left open, since Day 12 is explicitly the day for closing exactly this kind of thing.

Closing the Gap — Sequence Detector Feature Table

Feature	Req-ID	Category	Checker	Stimulus	Priority	Coverage Item
reset_returns_fsm_to_idle_and_detected_low	RST_001	RST	Checker w/ expected value	Directed	P0	Coverpoint on (state, detected) immediately after reset deasserts; bin reset_state = {IDLE, 0}.
basic_sequence_detected_correctly	FUNC_001	FUNC	Checker w/ expected value (exact cycle is hand-predictable, per Day 4: pulse in cycle 8, state S1011)	Directed	P0	Coverpoint on detected, bin fired={1}; coverpoint on state, all 5 bins (Day 11 model).
overlapping_matches_both_detected	CORNER_001	CORNER	Checker w/ expected value	Directed	P0	The “3-cycle gap” overlap bin from the Day 11 coverage model. P0, not just a corner case — overlap handling is the design’s literal, stated purpose (Day 4 goal: “detects the <i>overlapping</i> bit sequence”), so failing this means failing the design’s core requirement, not an edge case.
reset_mid_sequence_discards_partial_progress	CORNER_002	CORNER	Checker w/ expected value	Directed (reset asserted from each of S1/S10/S101, not just from IDLE)	P0	Cross coverage: state-at-reset-assertion × state-after-reset-deassertion; confirms reset was tested from every non-IDLE state, always landing on IDLE. Upgraded to P0 (not P1) — an async reset that doesn’t reliably work from every state is a fundamental reset-correctness failure, not a nice-to-have.
constant_din_never_falsely_detects	STRESS_001	STRESS	Assertion (detected implies previous-cycle state==S101 && din==1)	Directed — long constant-value run (all-0s, then all-1s), ≥20 cycles each	P1	Coverpoint on “constant-din-run length,” bin long_run = {≥20 cycles}, crossed with din value {0, 1}.
first_bit_after_reset_starts_progress_immediately	CORNER_003	CORNER	Checker w/ expected value	Directed	P2	Coverpoint on “first din value sampled after reset deasserts,” bins {0, 1}.

Closing the Gap — Traffic Light Feature Table

Feature	Req-ID	Category	Checker	Stimulus	Priority	Coverage Item
reset_forces_main_green_full_4_cycles	RST_001	RST	Checker w/ expected value	Directed	P0	Coverpoint on (state, count) immediately after reset deasserts; bin reset_state = {MAIN_GREEN, count==MAIN_GREEN_TIME-1} confirms reset loads MAIN_GREEN_TIME-1 (same as a reload), so the first phase is a full 4 cycles. This is the permanent regression test for ISSUE-D5-02 — the actual bug (reset loading the full value → 5-cycle first phase) now has a named, tracked test, not just a one-time fix.
full_cycle_sequence_correct	FUNC_001	FUNC	Checker w/ expected value	Directed	P0	State+duration coverpoint — all 4 states, each hit with its correct hold-length (4, 2, 6, 2).
wraparound_reloads_full_duration_every_time	CORNER_001	CORNER	Checker w/ expected value	Directed — must run at least 2 full loops	P0	Coverpoint crossing “loop iteration (1st vs. 2nd+)” with “MAIN_GREEN duration observed”; bin second_loop_duration={4} kept <i>distinct</i> from the first loop’s, and both compared against the exposed count. Note: the two-loop check alone did not catch ISSUE-D5-02 — the mis-sampled smoke log (ISSUE-D5-03) made the buggy 5-cycle first phase and the correct 4-cycle later phase both print as 4, so they matched. What actually caught it was honest per-cycle sampling plus the count column; this feature encodes checking the duration numerically (not by row-count) so the regression can’t hide the same way.
mutual_exclusion_of_green_lights	CORNER_002	CORNER	Assertion (!(main_green && side_green), every cycle)	N/A — free-running, no input to this design	P0	Cross of (main_green, side_green); bin illegal_both_green={1,1}. Unusual nuance, worth calling out: for a “must never happen” property, hitting this bin would mean a <i>failure</i> , not progress toward coverage closure — the goal is confirming the assertion had many opportunities to fire and never did, not filling the bin.
yellow_phase_other_road_stays_red	CORNER_003	CORNER	Assertion	N/A	P1	Cross of (main_yellow, side_red) and (side_yellow, main_red); bins confirm both yellow phases were entered with the correct partner-road state each time.
one_hot_lights_per_road	CORNER_004	CORNER	Assertion	N/A	P1	Per-road coverpoint on which single light is active. Likely redundant with FUNC_001’s state coverage — noted rather than silently duplicated (Day 9’s “not too fine” principle applies to coverage bins too, not just features).

12.2 Verification Metrics & Closure Thinking

Are all P0 features covered? Yes, now that the gap above is closed — every P0 feature across all three designs (FIFO: 7 of 8; Sequence Detector: 4 of 6; Traffic Light: 4 of 6) has a checker type, stimulus type, and coverage item assigned. Before this Day 12 pass, 12 of the 20 total features (all 6 of the Sequence Detector’s and all 6 of the Traffic Light’s) had *none* of these — including eight P0 features (four per design) and the permanent regression test for an actual, previously-found bug ([ISSUE-D5-02](#)).

Are all corner cases from the specs captured as Req-IDs? Now yes. Before this pass: FIFO’s were (Day 9), but the Sequence Detector’s 4 and Traffic Light’s 4 corner cases from the Day 7 spec work were sitting only as prose in [day7_spec_materials.md](#), never converted into the same Req-ID structure the FIFO received.

Are there any features with no coverage item assigned? No orphans remain after this pass. One near-duplicate was found and flagged rather than hidden: Traffic Light’s [CORNER_004](#) coverage likely overlaps with [FUNC_001](#)’s.

What would a DV engineer need to do next? (Preview, per the assignment.) All of the checker types, stimulus plans, and coverage models across all three designs are currently *specifications* — plain-English and tabular descriptions of what a testbench should do. None of it has actually been coded as a real SystemVerilog testbench with an automated scoreboard, real covergroups, or SVA properties yet (the Days 4–6 smoke tests are directed, manually-checked-by-eye tests, not the automated checker infrastructure described here). The next step would be building that actual testbench code — reference models for the scoreboards, [covergroup/coverpoint/bins](#) SystemVerilog for every coverage item listed across Days 9–12, and [assert property](#) SVA for every item marked “Assertion” above — and running it to get real, measured coverage percentages instead of the currently-planned-but-unmeasured bins.

Self-Review Against the Evaluation Rubric

One honest sentence per criterion, per the assignment’s instruction.

Criterion	Weight	Honest self-assessment
Depth of understanding — can you explain every decision?	25%	Mostly yes — every RTL design choice, bug, and category assignment across all 12 days has a stated “why,” but two of those “whys” (Traffic Light CORNER_004 ’s possible redundancy, and the Sequence Detector’s STRESS vs. NEG categorization for constant- din) were genuine judgment calls I flagged as debatable rather than asserted with full confidence.
Three RTLs — correctness, clean code, spec quality	25%	All three RTLs are simulation-confirmed correct (Icarus Verilog, not literally EDA Playground) with real bugs found and fixed along the way rather than assumed-correct; the spec document (RTL_Specifications.docx) is complete for all three designs and was actually re-verified page-by-page after generation, not just assumed to have built correctly.
Study Block documentation — KB thorough and in your own words	20%	Every day’s KB entry has all four required parts (topics, concepts, experimental outputs, bugs/confusions) in original prose, not copied from the assignment text; the main risk here is length/density rather than missing content.
DV thinking — feature list, coverage model, test category reasoning	15%	Strong for the FIFO (8 refined features, full traceability) but was incomplete for the other two designs until this Day 12 pass — that gap, now closed, is itself the most honest evidence of where this criterion genuinely stood before today.
Issue log and reflection — honest, specific, detailed	10%	The Issue Log now has entries for every day something noteworthy occurred, including this day’s own gap-finding, rather than only logging the more flattering “bug found and fixed” stories.
Presentation and Q&A	5%	Not yet assessable — Days 13–14 (reflection and presentation) haven’t been reached yet in this program.

Repo Cleanliness

Experimental Work item: “Clean up the Git repo.”

Honest limitation: I don’t have access to your actual Git repository — I can produce and organize files, but I can’t literally run `git add` / `git commit` / clean up your working tree. Here’s what I *can* confirm from my side: every file I’ve generated across all 12 days is listed below, organized the way the assignment’s repo structure implies (`rtl/`, `docs/experiments/`, top-level KB and Issue Log). Nothing here is a stray/junk file — each one maps to a specific Output bullet from a specific day. You’ll still need to actually verify your own local repo matches this and commit it yourself.

Final Check — RTL_Specifications.docx Completeness

All 8 sections — Overview, Key Features, Interface Table, Microarchitecture, FSM Specification, Timing, Corner Cases, RTL Code — are present for all three designs, with no placeholder text. **Two follow-ups remain before the docx is truly consistent with the final RTL**, flagged here rather than silently left stale:

1. **The spec doc predates the RTL corrections and must be updated to match them.** Specifically: the Sequence Detector is now a **5-state Moore FSM** (`IDLE`, `S1`, `S10`, `S101`, `S1011`, `detected = (state==S1011)`), not a 4-state Mealy-with-registered-output — so the interface table, FSM section, state count, transition table (10 arcs), and the `detected` timing (pulse in cycle 8, in `S1011`) all need correcting. The Traffic Light reset now loads `MAIN_GREEN_TIME-1` (first phase a true 4 cycles), so any “loads the full duration at reset” wording must be fixed. These mirror the corrections already applied to the Day 2/4/5/7 markdown files and the Issue Log (`ISSUE-D2-01`, `ISSUE-D4-01`, `ISSUE-D5-02`, `ISSUE-D5-03`).
2. **The docx does not yet contain the new Req-ID/checker/coverage tables** built in this Day 12 pass for the Sequence Detector and Traffic Light.

Until item 1 is done, the spec doc is internally complete but **not consistent with the corrected RTL** — that reconciliation is the single most important remaining task for the graded deliverable.